

Enriching with Data Science

A Practitioner's Playbook
for Knowledge Graphs

dr Zbigniew Galar

Moving Beyond Queries to Structural Insight



Influence

Finding influential people and critical bridges in social graphs.



Vulnerability

Identifying critical junctions and single points of failure in physical networks.



Patterns

Uncovering coordinated cells of fraudsters or common pathogens in disease pathways.

Key Takeaway: Graph algorithms compute over the topology of your data, transforming raw connections into quantified insights.

The Analytical Toolkit

Network Propagation



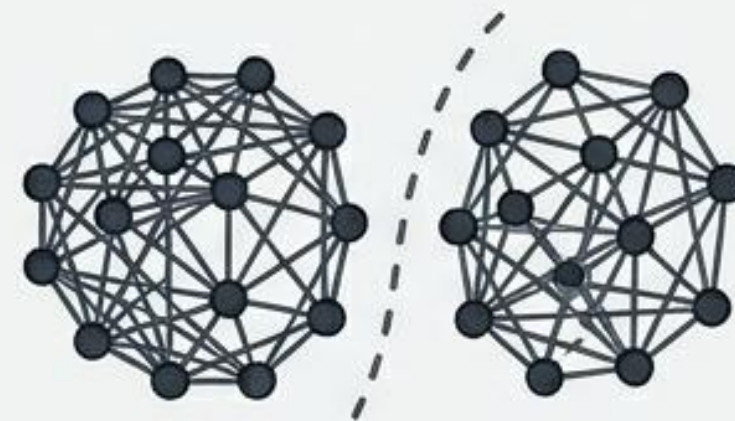
Tracing deep pathways to track disease spread or supply-chain weaknesses.

Influence



Calculating centrality to find bridge nodes that spread information or disruption quickly.

Community Detection



Partitioning networks by removing weak links to find tight-knit communities or fraud rings.



Similarity

Identifying nodes with common properties or structural hierarchies (e.g., for recommendations).

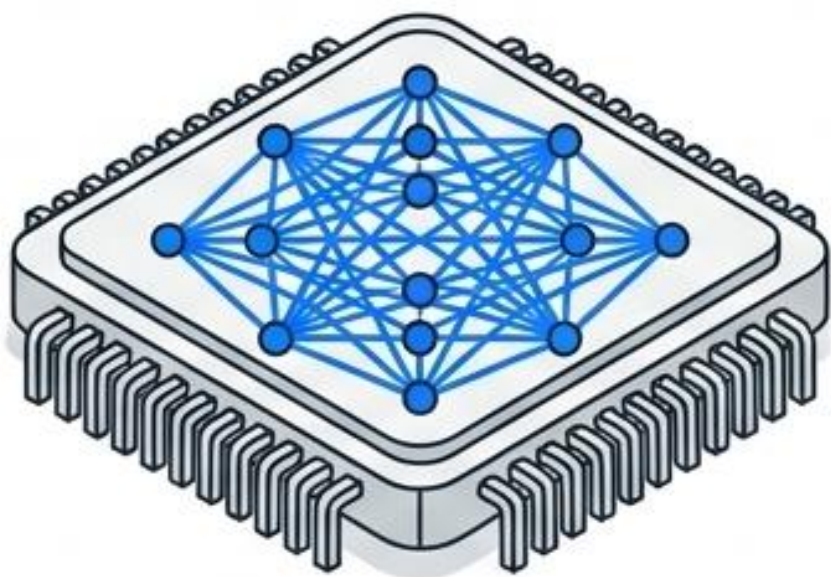


Link Prediction

Computing missing relationships based on existing topology (e.g., triangle completion).

The Physics of Graph Computation

Scale-Up (In-Memory CPU/RAM)

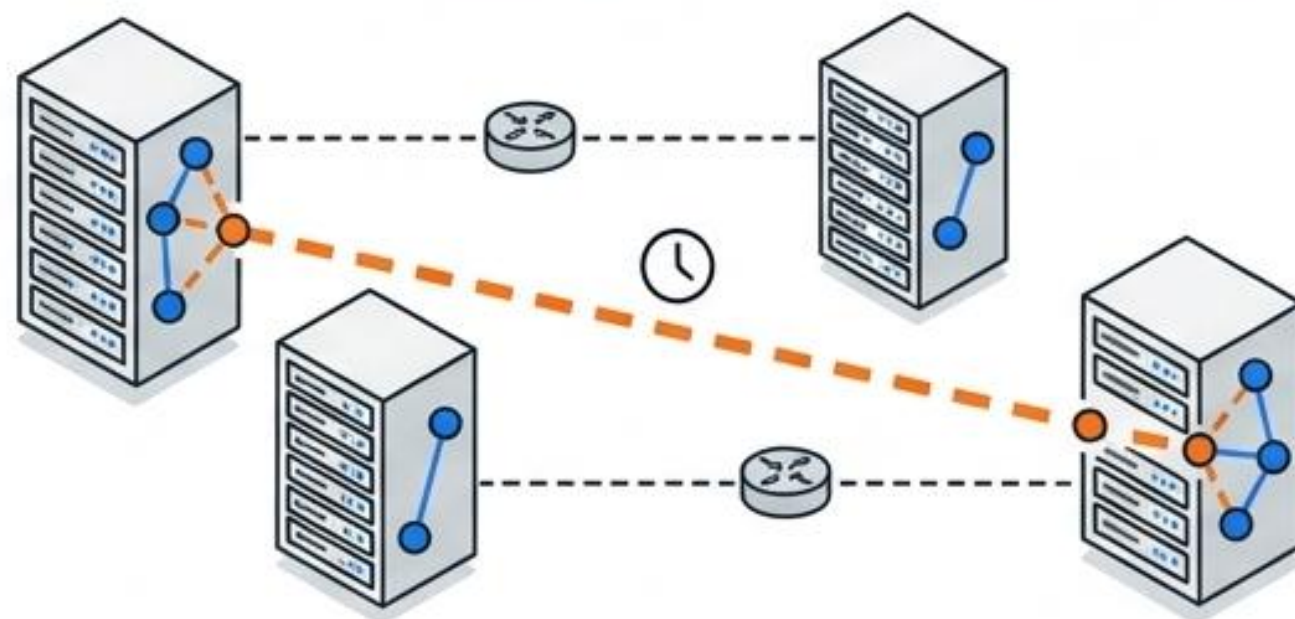


Architecture: Single large memory space.

Locality: Neighbors of any node are in local memory.

Performance: Simple pointer dereferencing.
Best-in-class speed for non-linear algebra problems.

Scale-Out (Distributed Clusters)

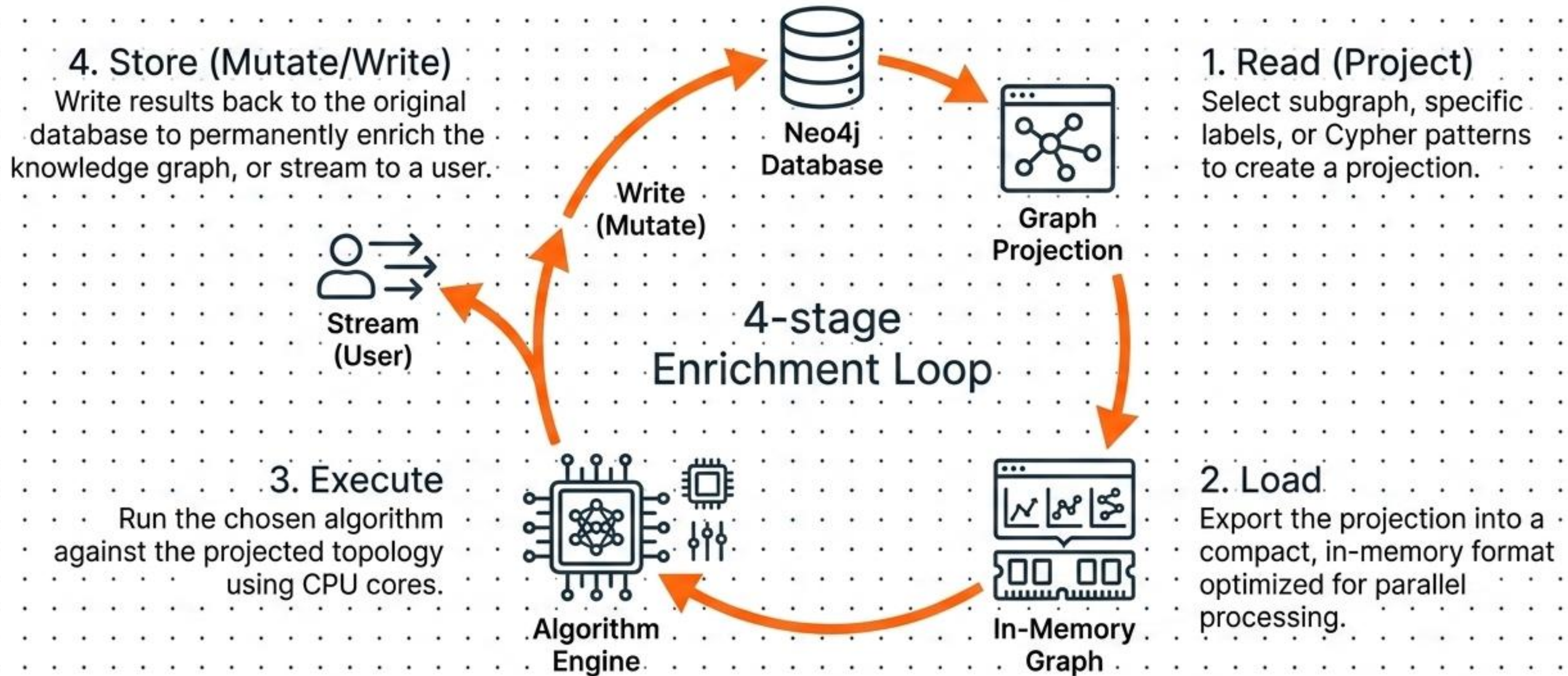


Architecture: Nonuniform architecture across multiple servers.

Locality: Neighbors reside on remote servers.

Performance: Pointer dereferencing is replaced by expensive, high-latency network operations.

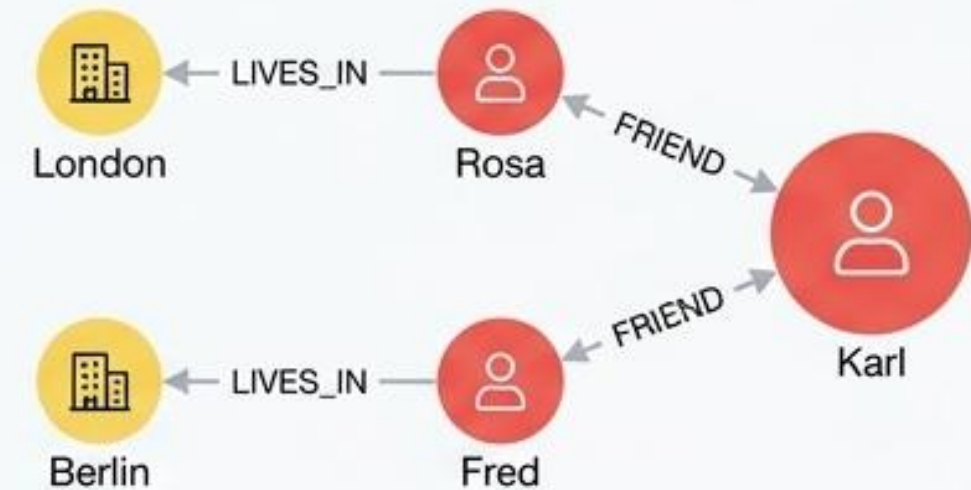
The Four-Phase Execution Lifecycle



Execution in Cypher: Betweenness Centrality

```
CALL gds.betweenness.write('gds-example-graph',  
{writeProperty:'betweennessCentrality'});
```

```
+-----+  
| "n" |  
+-----+  
| {"country":"UK","city":"London"} |  
| {"betweennessCentrality":0.0,"name":"Fred"} |  
| {"betweennessCentrality":2.0,"name":"Karl"} |  
| {"betweennessCentrality":0.0,"name":"Rosa"} |  
| {"country":"DE","city":"Berlin"} |  
+-----+
```



Karl connects Rosa and Fred. He holds the highest centrality score (2.0). Without him, the social network collapses. Node size is dynamically mapped to the computed centrality score.

Transitioning to the Data Lab

Abstracting database complexity for rapid hypothesis testing
using the Neo4j GDS Python driver.

```
from graphdatascience import GraphDataScience  
gds = GraphDataScience("bolt://127.0.0.1:7687", auth=("neo4j", "yolo"))
```

The Pythonic API allows **data scientists** to interact with the graph entirely via native Python objects, seamlessly combining run_cypher data loading with in-memory algorithm execution.

Case Study: The Great Britain Rail Network

A highly interconnected, real-world physical dataset loaded directly from CSVs into Neo4j.



Nodes

`(:Station {name, crs})`
representing physical stations.

Relationships

`-[:TRACK {distance}]->`
representing track length
between stations, stored as a
floating-point value.

Establishing Baseline Topologies: Pathfinding

Creating a specialized trains projection (station names + distances) to run routing algorithms.

```
shortest_path = gds.shortestPath.dijkstra.stream(  
    gds.graph.get("trains"),  
    sourceNode=bham, targetNode=eboro,  
    relationshipWeightProperty="distance")
```

Result: Shortest distance: 298.0
(Birmingham New Street to Edinburgh)



The Hypothesis: Identifying the Critical Hub

The Assumption

Birmingham New Street is the most vital station in the network. It requires the highest investment in infrastructure redundancy.

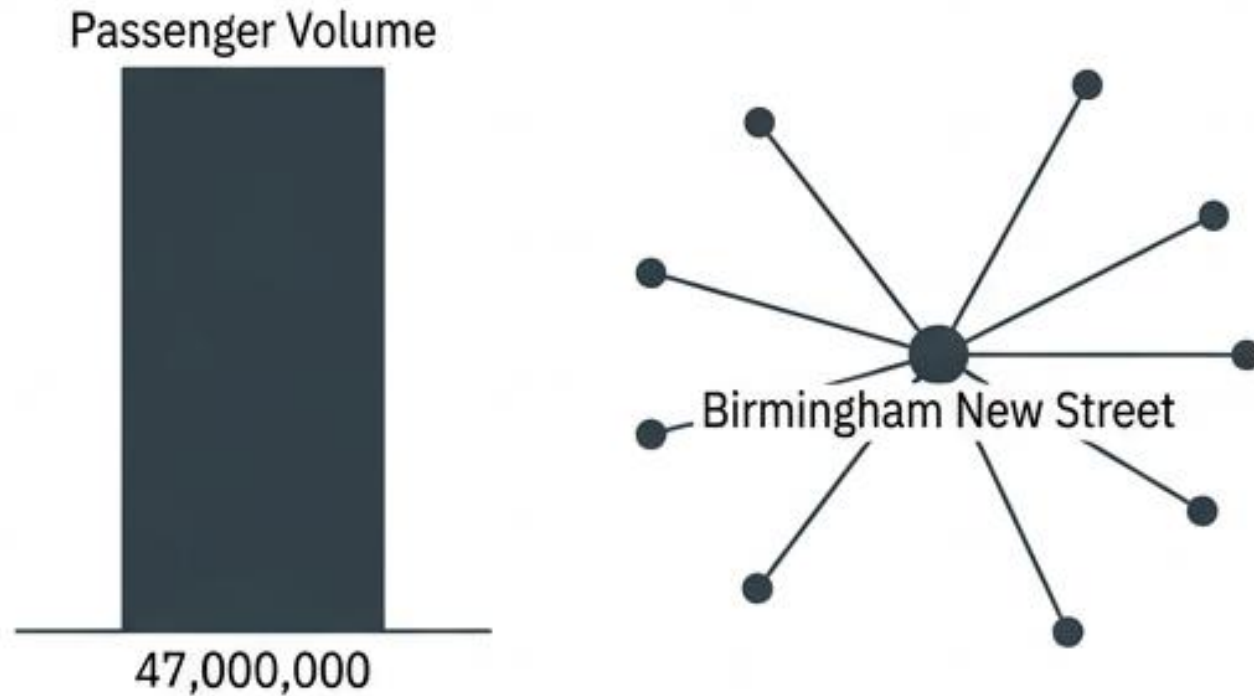
**Birmingham
New Street**

The Evidence (Human Logic)

- Geographically centered in England. Busiest
- Busiest station outside London.
- Passenger volume: 47 million per year (+ 7 million interchanges).

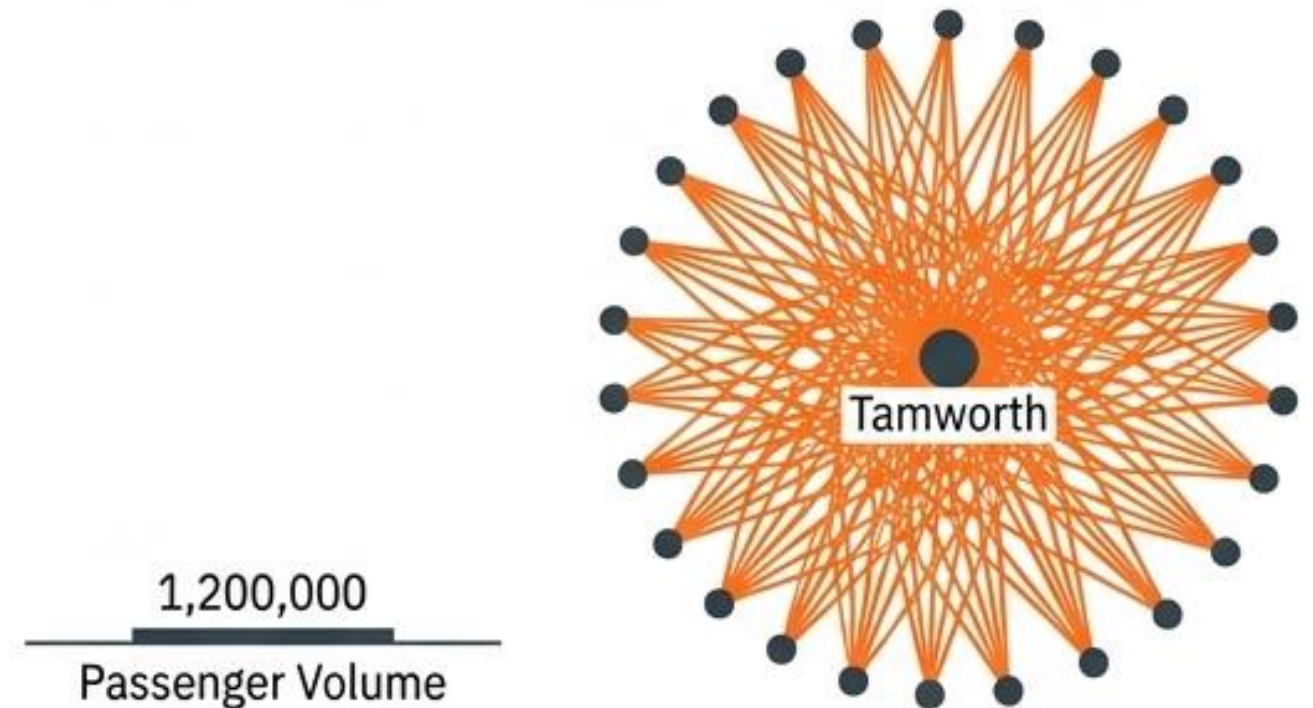
Expectation vs. Reality: The Centrality Test

Birmingham New Street (The Expectation)



Passengers: 47,000,000
Betweenness Centrality Score: 254,706

Tamworth (The Reality)



Passengers: 1,200,000
Betweenness Centrality Score: 1,967,643

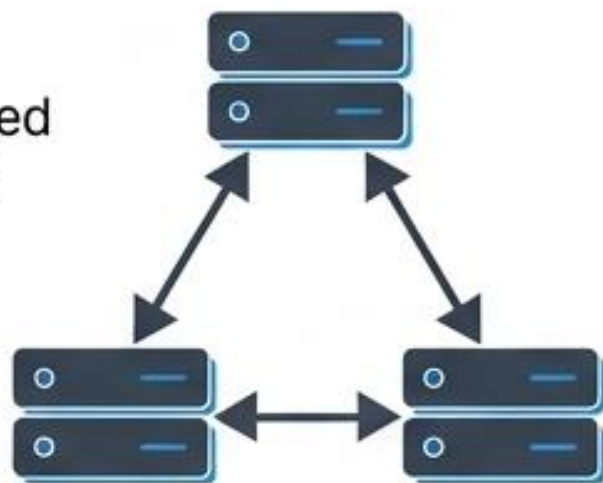
The Insight: Tamworth's score is nearly 8x higher. While passenger volume is low, **Tamworth is the true physical bottleneck**. A track failure here ripples through the entire Great Britain network.

Enterprise Productionization: HTAP Architecture

Primaries

High I/O, Modest CPU

Dedicated purely to clustered transaction processing and Cypher queries.



Secondaries

Fast multi-core CPUs, High RAM

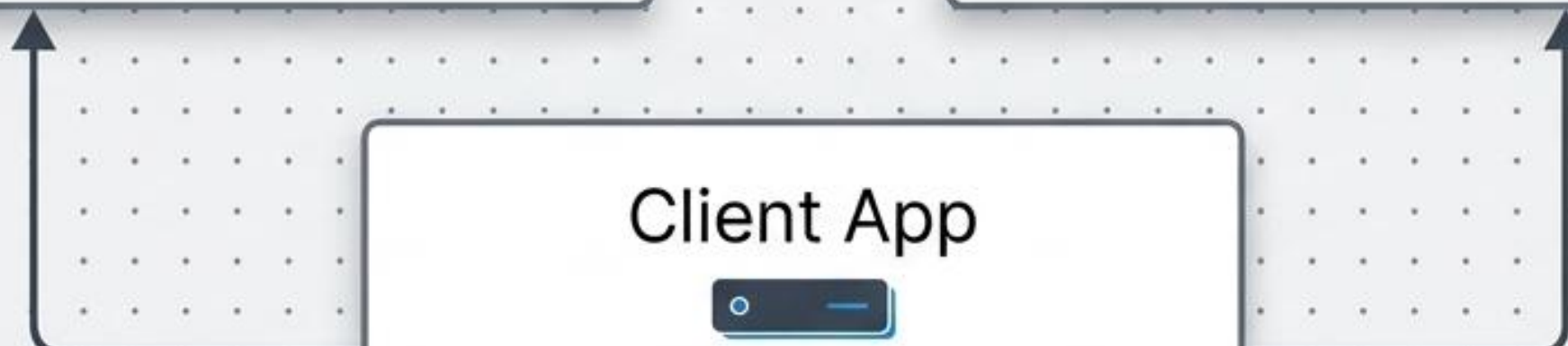
Dedicated to compute-intensive graph data science and global queries. Asynchronous updates prevent transactional slowdown.



Client App



Transparently routes jobs to the appropriate physical server based on workload tagging.



Enforcing Data Freshness: The Causal Barrier

The Challenge: Secondaries update asynchronously. How do we guarantee an algorithm runs on fresh data?

Causal Barrier Flowchart

Causal Barrier

Step 1: Commit

Transaction commits on Primary.



Step 2: Capture

```
String bookmark =  
session.lastBookmark();
```

Captures the exact transaction ID.

Step 3: Block



```
session.beginTransaction(  
    bookmark )
```

Pass the bookmark when opening the GDS session.

Step 4: Execute

The Secondary waits to process the transaction before running the algorithm, ensuring you "read your own writes."



Closing the Loop: Permanent Enrichment

Experimentation Mode (**.stream**)

```
result = gds.betweenness.stream(graph)
```

Returns insight to the Jupyter notebook console for the human analyst.

Production Mode (**.write**)

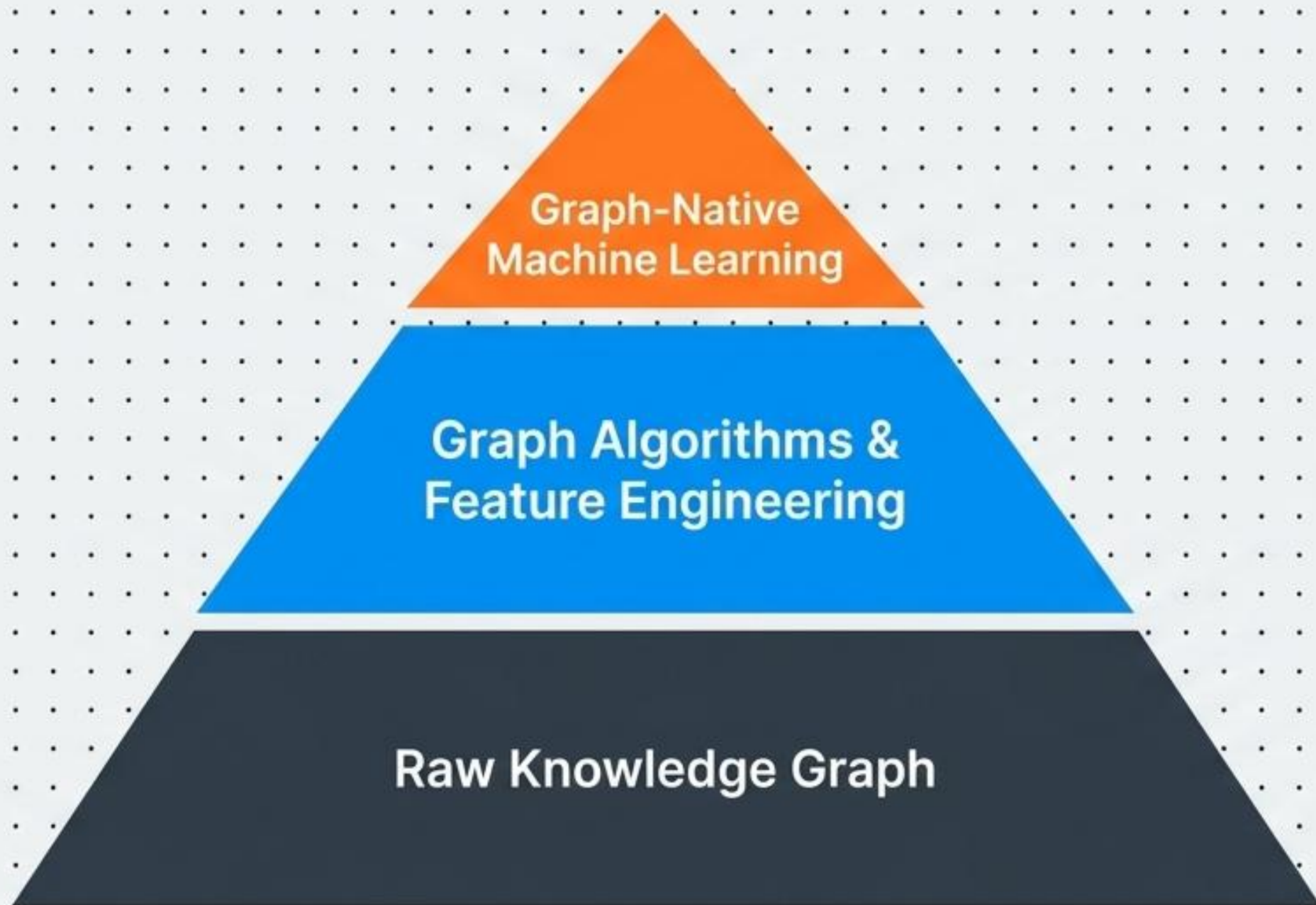
```
result = gds.betweenness.write(graph,  
    writeProperty="betweenness")
```

Permanent Enrichment

Writes the 1,967,643 centrality score directly onto the Tamworth node as a permanent property.

Result: 2,593 stations processed and permanently enriched. The knowledge graph is now smarter than when we started.

The Foundation for Predictive Models



The Takeaway

Graph algorithms do more than surface human insights. By continuously running `.write()` operations, we engineer rich, topological features.

Next Steps

These computed structural metrics—like betweenness centrality and community partitions—become the definitive training features for graph-native Machine Learning models.